

Guía de trabajo del equipo — Scaps

Cómo trabajamos y qué hay para hacer en el Sprint 0

Scaps es nuestro e-commerce de gorras con visor de productos en 3D. Esta guía explica, en pocos minutos, cómo nos organizamos día a día y cuáles son las tareas del primer sprint. Léela antes de tomar tu primera tarea.

Cómo nos organizamos

Trabajamos con un tablero de tareas en **GitHub Projects**, no con roles fijos. No hay "dueños" de áreas: el trabajo está en un backlog priorizado y **cada uno toma la tarea que sigue**. Una tarea avanza por estas cinco columnas, de izquierda a derecha:



- **Backlog** — todo lo que hay que hacer pero todavía no entró al sprint.
- **To Do** — listo para tomar. Tareas priorizadas que ya se pueden agarrar.
- **In Progress** — lo que alguien está haciendo ahora. Una sola por persona.
- **In Review** — terminado, esperando que otro lo revise (Pull Request).
- **Done** — hecho, revisado y mergeado.

El flujo, paso a paso

Cuando te vas a poner a trabajar:

1. **Entrás al tablero** (GitHub Projects) y mirás la columna **To Do**.
2. **Tomás la tarea de más arriba**. Están ordenadas por prioridad: no elijas la más fácil del medio.
3. **La movés a In Progress** y la **convertís en issue** (clic en la tarjeta → *Convert to issue*). Le ponés los **labels** que figuran en el backlog (área + prioridad) y **te la asignás**, para que el resto vea que la estás haciendo.
4. **Creás una rama** desde `main` para esa tarea (por ejemplo `feat/scaffold-frontend`).
5. **Programás** y vas guardando tus commits en esa rama.
6. Cuando terminás —y cumple el "*hecho cuando*"— **abrís un Pull Request** y en la descripción escribís `Closes #N` (el número del issue).
7. **Otro del equipo lo revisa** y aprueba. Si pide cambios, los hacés y volvés a pedir revisión.
8. **Se mergea el PR**. El issue se cierra y la tarjeta **pasa sola a Done**: no tenés que mover nada a mano.

Las reglas (pocas y simples)

- **Una sola tarea en curso por persona**. Terminá lo que empezaste antes de tomar otra.
- **Tomá de arriba hacia abajo** en To Do. Lo prioritario primero.
- **No se pushea directo a `main`**. Todo entra por Pull Request con al menos una aprobación.

- **Si te trabás más de un día**, marcá la tarjeta como bloqueada y pedí ayuda. Una tarea crítica frenada en silencio es lo peor que puede pasar.
- **El que revisa** controla que la tarea cumpla su *"hecho cuando"* y que no rompa nada.

Backlog del Sprint 0

El Sprint 0 es preparación: dejar el proyecto listo para empezar a construir. Cada tarea indica su área entre corchetes, qué hacer concretamente y, cuando aplica, el criterio para darla por terminada (*hecho cuando*).

To Do — se pueden tomar ya:

- **[visor-3d] Conseguir modelos .glb de un banco gratuito** — bajar varios modelos con licencia libre (CC0) de un banco como Sketchfab o Poly Pizza para el catálogo de demo; que se lean como gorras y chequear que la licencia permita el uso. *Hecho cuando*: hay modelos .glb usables en el repo.
- **[auth] Confirmar requisitos de backend/auth con la cátedra** — preguntar si hay requisitos sobre el backend o la autenticación. Ya decidimos construir backend (NestJS) y auth (JWT + roles) propios, así que es solo para confirmar; no bloquea. (*Consulta.*)
- **[setup] Inicializar el monorepo con workspaces** — crear la estructura con npm workspaces (`apps/web` y `apps/api`) y el `package.json` raíz apuntando a `apps/*` . Sin `packages/shared` , sin Turborepo ni Nx. *Hecho cuando*: instalar desde la raíz resuelve los workspaces y está mergeado en `main` .
- **[infra] Proteger la rama main** — crear una branch protection rule en `main` : exigir PR con al menos 1 aprobación y bloquear el push directo. (*Configuración en GitHub.*)
- **[setup] Diseñar el modelo de datos (ERD)** — diagramar las entidades del MVP (usuarios, roles, productos con "destacado", imágenes y modelo .glb , stock, carrito, órdenes) con sus campos y relaciones. *Hecho cuando*: hay un ERD acordado y guardado en la carpeta de docs.
- **[setup] Definir el contrato de la API** — listar los endpoints del MVP con método, ruta, request y response (productos, auth, carrito, órdenes). Es lo que permite a front y back trabajar en paralelo. *Hecho cuando*: hay un documento acordado en el repo. (*Depende del ERD.*)
- **[setup] Diseñar los wireframes** — bocetar (sin diseño fino) las pantallas: landing (visor + CTA al catálogo), catálogo en cards, ficha de producto (galería de imágenes + opción Ver en 3D), carrito, login y dashboard. *Hecho cuando*: son accesibles para todo el equipo.

Backlog — esperan que se libere una dependencia:

- **[setup] Configurar ESLint + Prettier compartidos** — reglas de lint y formato compartidas para `web` y `api` desde la config raíz, con scripts de lint y formato. *Hecho cuando*: `lint` corre en ambos workspaces y el formateo automático funciona. (*Depende del monorepo.*)
- **[setup] Scaffold del frontend** — crear la app en `apps/web` con React + Vite + TypeScript y Tailwind configurado. *Hecho cuando*: `npm run dev` levanta una página inicial con estilos de Tailwind. (*Depende del monorepo.*)
- **[setup] Scaffold del backend** — crear la app en `apps/api` con NestJS + TypeScript y un endpoint de health check. *Hecho cuando*: el backend levanta en local y `GET /health` responde 200. (*Depende del monorepo.*)
- **[infra] Configurar CI en GitHub Actions** — un workflow que en cada PR corra lint + build de `web` y `api` . *Hecho cuando*: el check de CI aparece en los PR y queda en verde. (*Depende de los scaffolds.*)

- **[infra] Variables de entorno + .env.example** — definir las variables de `web` y `api`, dejar un `.env.example` sin secretos reales y asegurar que `.env` esté en `.gitignore`. *Hecho cuando:* existe `.env.example` y `.env` está ignorado. *(Depende de los scaffolds.)*
- **[documentation] README inicial** — descripción del proyecto y pasos para levantarlo en local (requisitos, instalar, correr `web` y `api`). *Hecho cuando:* alguien que clona el repo puede levantarlo siguiendo solo el README. *(Depende del monorepo.)*
- **[visor-3d] POC del visor 3D** — con React Three Fiber + drei, mostrar un modelo en pantalla y poder rotarlo con el mouse (OrbitControls). *Hecho cuando:* en local se ve el `.glb` y se puede rotar. *(Depende del scaffold del frontend y de los modelos del banco.)*